Challenge Name: Some Things Are Only Safe Because They Were Never Supposed to Happen Twice

Summary

The service exposed an ECDSA signing API on secp256k1 and a privileged `/api/redeem` endpoint that accepted signed maintenance commands. The command `give_flag` was blocked from the signing endpoint, so the goal was to forge a valid ECDSA signature for it.

The vulnerability was ECDSA nonce reuse. After collecting signatures from the signing oracle, two different messages had the same `r` value. In ECDSA, a repeated `r` normally means the same nonce `k` was reused. With two signatures using the same nonce, the nonce and private key can be recovered algebraically.

The recovered flag is intentionally excluded from this report.

Target

```
https://667e644d-3cf8-4dcc-b303-3b6a2e3fa66d.222.255.138.122.nip.io/
```

Important endpoints:

```
GET  /api/pubkey
GET  /api/sign
POST /api/redeem
```

Solve Flow

1. Opened the web portal and identified ECDSA over secp256k1.
2. Fetched the public key from `/api/pubkey`.
3. Requested signatures from `/api/sign`.
4. Grouped signatures by `r`.
5. Found two different messages with the same `r`.
6. Computed SHA-256 hashes of both signed messages.
7. Recovered the repeated nonce `k`.
8. Recovered the private key `d`.
9. Verified that `d*G` matched the published public key.

10. Signed the privileged command `give_flag`.
11. Submitted the forged signature to `/api/redeem`.
12. The server returned HTTP 200 and an authorized response.

Public Key

The public key endpoint returned:


```
{
  "Qx": "0x6c8cf8c240f5c1166eb29ce60ba2c3b6251086abe3bc58b450aacdc1a5976707",
  "Qy": "0xc9f6fa0be275581f34a75b1ecdc1718d97d377ab3f72dc5f1260a08904e88ee4",
  "curve": "secp256k1"
}
```


Repeated Nonce Evidence

Two signed messages reused the same `r`:


```
{
  "message": "{\"id\":24,\"model\":\"nova-
1\",\"output\":\"6a689f3a03ffa503\"}",
  "r": "0x37a59aab5b01c9f0eca1b6dfa2744da4d6493a774aab45dcc8d52bb1065f0955",
  "s": "0x80073e737b321da1fb97c2e4dc2dd4114af566dce3249d075bc528024bbd188d"
}
```

```
{
  "message": "{\"id\":104,\"model\":\"nova-
1\",\"output\":\"991329a5afd9009e\"}",
  "r": "0x37a59aab5b01c9f0eca1b6dfa2744da4d6493a774aab45dcc8d52bb1065f0955",
  "s": "0x406f344fe979b5c92703ad05c1bbbda9779b426781969d029a1f9d25433f316d"
}
```


The messages are different, but `r` is identical. This is the critical failure.

Vulnerability

ECDSA signing uses:

```
r = (k*G).x mod n
s = k^-1 * (z + r*d) mod n
```

Where:

```
k = per-signature nonce
d = private key
z = message hash
n = curve group order
```

For two signatures with the same nonce:

```
s1 = k^-1 * (z1 + r*d) mod n
s2 = k^-1 * (z2 + r*d) mod n
```

Subtract:

```
s1 - s2 = k^-1 * (z1 - z2) mod n
```

Recover the nonce:

```
k = (z1 - z2) * (s1 - s2)^-1 mod n
```

Recover the private key:

```
d = (s1*k - z1) * r^-1 mod n
```

Some ECDSA implementations normalize `s` to low-S form. A robust solve should try both `s` and `n-s` for each signature. In this case, the direct pair worked.

Final Solve Script

```python
#!/usr/bin/env python3
import hashlib
import json
import secrets
import ssl
import urllib.request

BASE = "https://667e644d-3cf8-4dcc-b303-3b6a2e3fa66d.222.255.138.122.nip.io"
ctx = ssl._create_unverified_context()

p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
```

```
def inv(x, mod):
    return pow(x % mod, -1, mod)


def add(P, Q):
    if P is None:
        return Q
    if Q is None:
        return P
    x1, y1 = P
    x2, y2 = Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return None
    if P == Q:
        lam = (3 * x1 * x1) * inv(2 * y1, p) % p
    else:
        lam = (y2 - y1) * inv(x2 - x1, p) % p
    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return x3, y3


def mul(k, P=(Gx, Gy)):
    R = None
    Q = P
    k %= n
    while k:
        if k & 1:
            R = add(R, Q)
        Q = add(Q, Q)
        k >>= 1
    return R


def z(message):
    return int.from_bytes(hashlib.sha256(message.encode()).digest(), "big")


def sign(message, d):
    zz = z(message)
    while True:
        k = secrets.randbelow(n - 1) + 1
        R = mul(k)
        r = R[0] % n
```

```
        if r == 0:
            continue
        s = inv(k, n) * (zz + r * d) % n
        if s == 0:
            continue
        if s > n // 2:
            s = n - s
        return r, s


def get_json(path):
    with urllib.request.urlopen(BASE + path, context=ctx, timeout=10) as r:
        return json.loads(r.read())


def find_repeated_r(limit=500):
    seen = {}
    for _ in range(limit):
        sig = get_json("/api/sign")
        r = int(sig["r"], 16)
        if r in seen and seen[r]["message"] != sig["message"]:
            return seen[r], sig
        seen[r] = sig
    raise RuntimeError("no repeated r found")


pub = get_json("/api/pubkey")
Q = (int(pub["Qx"], 16), int(pub["Qy"], 16))

a, b = find_repeated_r()
r = int(a["r"], 16)
s1_raw = int(a["s"], 16)
s2_raw = int(b["s"], 16)
z1 = z(a["message"])
z2 = z(b["message"])

private_key = None
for s1 in {s1_raw, (-s1_raw) % n}:
    for s2 in {s2_raw, (-s2_raw) % n}:
        if s1 == s2:
            continue
        k = (z1 - z2) * inv(s1 - s2, n) % n
        d = (s1 * k - z1) * inv(r, n) % n
        if mul(d) == Q:
            private_key = d
            break
```

```
    if private_key is not None:
        break

if private_key is None:
    raise RuntimeError("failed to recover private key")

rr, ss = sign("give_flag", private_key)
body = json.dumps({
    "message": "give_flag",
    "r": hex(rr),
    "s": hex(ss),
}).encode()

req = urllib.request.Request(
    BASE + "/api/redeem",
    data=body,
    method="POST",
    headers={"Content-Type": "application/json"},
)

with urllib.request.urlopen(req, context=ctx, timeout=10) as r:
    print(r.status)
    print(r.read().decode())
```